1 Slide 1: Proposal Title

Slide Title: *Evaluating FedAvg and FedProx for Federated Crop Disease Detection under Data Heterogeneity*
Suggested Timing: 35–45 seconds.

What to Say in the Presentation (Literal Script)

"Good morning [afternoon], distinguished committee members and colleagues.
Today, I present my research project proposal titled: **'Evaluating FedAvg and FedProx for Federated Crop Disease Detection under Data Heterogeneity: A Simulated Collaborative Agricultural Network with Non-IID Foliar Distributions'**.
This work lies directly at the intersection of **Distributed Systems** and applied machine learning. We design and simulate an environment where multiple agricultural farms cooperate as autonomous compute nodes to train a global crop diagnostic model, without ever centralizing their private data, and formally addressing the challenges of latency, statistical heterogeneity, and fault tolerance."

Transition: *"To understand the critical need for this distributed architecture, let us first examine the agronomic problem and the bottlenecks of traditional centralized approaches."*

---

2 Slide 2: Real-World Motivation: Agricultural Impact & Data Silos

Slide Title: *Real-World Motivation: Agricultural Impact & Data Silos*
Suggested Timing: 50–60 seconds.

Key Distributed Systems Points to Emphasize

- **Failures of centralized cloud computing:** Excessive traffic over weak, unstable rural WAN links.
- **Data sovereignty and silos:** Commercial privacy of agricultural producers.
- **Federated Learning (FL) paradigm:** Moving computation to where data resides, rather than data to computation.

What to Say in the Presentation (Literal Script)

"Globally, foliar crop diseases cause catastrophic annual yield losses: between **20% and 40% of global crop yields**, according to research published in *Nature Ecology & Evolution*. Early computer vision-based diagnosis can cut **unnecessary chemical pesticide misuse by up to 50%**.
However, in real-world agricultural settings, **centralized cloud architectures break down** due to two fundamental distributed systems barriers:

1. **Commercial confidentiality and sovereignty:** Photographic records of crop pests and yields are private strategic assets; farmers strictly refuse to upload them to third-party central repositories.
2. **Rural WAN bottlenecks:** Farms operate over satellite or 3G/4G cellular connections with severely limited bandwidth; uploading gigabytes of high-resolution imagery is both economically and technically infeasible.

Our solution: We deploy **Federated Learning (FL)**. We decouple the training process: raw images remain locally on the farms, and only serialized numerical parameters travel across the network."

Transition: *"However, decentralizing the learning process introduces a major theoretical hurdle inherent to distributed data: statistical heterogeneity, or the Non-IID phenomenon."*

---

3 Slide 3: The Distributed Challenge: Non-IID Foliar Skew & Client Drift

Slide Title: *The Distributed Challenge: Non-IID Foliar Skew & Client Drift*
Suggested Timing: 55–65 seconds.

Key Distributed Systems Points to Emphasize

- **Statistical heterogeneity (Non-IID):** Asymmetric local data distributions across individual nodes.
- **Client Drift:** Local gradients diverge toward conflicting local optima.
- **Destructive interference in FedAvg:** Catastrophic degradation on minority classes and inter-round oscillations.

What to Say in the Presentation (Literal Script)

"In a real-world distributed system, agricultural data is **never identically distributed (Non-IID)**. As shown on the slide:

- **Farm A**, situated in a humid valley, primarily concentrates necrotic blights and rusts.
- **Farm B**, in a cooler highland region, exhibits mostly bacterial spot pathologies.
- **Farm C**, in the lowlands, gathers viral infections alongside healthy foliage.

Compounding this is *Covariate Shift*: differing sunlight angles, soil backgrounds, and mobile camera sensors.
The critical distributed systems consequence is Client Drift: When each farm runs multiple local SGD epochs, its model weights converge toward mutually incompatible local minima. When naively averaging these weights using the classical **FedAvg** algorithm, the client gradients interfere destructively: the system suffers from sluggish convergence, severe oscillations, and worst of all, the **catastrophic forgetting of minority crop diseases**."

Transition: *"To address this distributed pathology, we formulate our core research question and formal testable hypotheses."*

---

4 Slide 4: Research Question & Formal Hypotheses

Slide Title: *Research Question & Formal Hypotheses*
Suggested Timing: 50–60 seconds.

What to Say in the Presentation (Literal Script)

"Our central research question is formulated as follows:

> *'To what extent does the FedProx algorithm improve diagnostic performance, consensus stability, and cross-client generalization compared to FedAvg in a three-node distributed agricultural system under heterogeneous (Non-IID) data distributions?'*

To rigorously investigate this, we propose four testable hypotheses:

- **Hypothesis 1 (Performance Equity):** Under homogeneous (IID) data, FedAvg and FedProx will achieve parity; however, under severe Non-IID skew, FedProx will outperform FedAvg in Macro-F1 score and safeguard recall on rare diseases.
- **Hypothesis 2 (Optimization Stability):** FedProx will dampen inter-round oscillations via its proximal penalty, achieving a significantly lower inter-round variance ($S_{\Delta acc}$).
- **Hypothesis 3 (μ Sensitivity):** There exists an optimal intermediate value of $\mu \in [0.01, 0.1]$; an excessively large μ (≥ 1.0) will overly constrain and freeze local learning.
- **Hypothesis 4 (Fault Tolerance / Node Churn):** In the event of an abrupt node disconnection midway through training, FedProx will preserve greater consensus stability and diagnostic resilience than FedAvg."

Transition: *"Let us now examine the formal distributed systems foundations, communication protocols, and mathematical optimization underpinning our architecture."*

5 Slide 5: Distributed Foundations: Protocols & Optimization

Slide Title: *Distributed Foundations: Protocols & Optimization*
Suggested Timing: 55–65 seconds.

Key Distributed Systems Points to Emphasize

- **Star topology and RPC messages:** Communication governed by `FitIns` and `FitRes` primitives.
- **Synchronous barrier model ($T^{(t)}_{\text{round}}$):** Straggler dynamics in edge networks.
- **Communication complexity:** Over 98% traffic reduction by transmitting 32-bit parameters ($B_{\text{round}} = 8Kd$ bytes).
- **FedProx formulation:** Proximal term $\frac{\mu}{2}\|w - w^{(t)}\|_2^2$ acting as an elastic centripetal force.

What to Say in the Presentation (Literal Script)

"On this slide, we formalize our distributed systems framework and mathematical optimization:

1. Distributed Coordination and Networking:

- We operate over a **star topology** where a central coordinator orchestrates worker nodes using two core message primitives: `FitIns` (fit instruction delivering global weights) and `FitRes` (fit result returning updated weights and sample counts).
- Communication complexity is reduced by over **98%** by transmitting only $8Kd$ bytes per round rather than raw high-resolution foliar photography.

2. Optimization Consensus Algorithms:

- **FedAvg (Baseline):** Computes a sample-weighted parameter aggregation: $w^{(t+1)} = \sum \frac{n_k}{n} w_k^{(t+1)}$.
- **FedProx (Investigated):** Introduces a quadratic proximal penalty $\frac{\mu}{2}\|w - w^{(t)}\|_2^2$. Mathematically, this acts as an 'elastic spring' anchoring local client updates to the global centroid, preventing local gradients from collapsing into idiosyncratic local farm optima."

Transition: *"To understand exactly how synchronization round time is governed, let us break down the barrier latency equation on the next slide."*

6 Slide 6: Understanding the Barrier Latency Formula

Slide Title: *Understanding the Barrier Latency Formula*
Suggested Timing: 55–65 seconds.

Key Distributed Systems Points to Emphasize

- **Synchronous barrier synchronization:** Round duration $T_{\text{round}}^{(t)}$.
- **The** max **operator:** The slowest client (straggler) dictates round completion time.
- **Component breakdown:** Network downlink (T_{down}), local compute (T_{comp}), network uplink (T_{up}), and server aggregation (T_{agg}).
- **Client compute heterogeneity:** Why GPU vs. CPU worker hardware creates straggler bottlenecks.

What to Say in the Presentation (Literal Script)

"On Slide 6, we break down the formal barrier latency equation that governs wall-clock execution time in distributed learning:

$$T_{\text{round}}^{(t)} = \max_{k \in \mathcal{K}_t} \left\{ T_{\text{down}}^{(k)} + T_{\text{comp}}^{(k)} + T_{\text{up}}^{(k)} \right\} + T_{\text{agg}}$$

As detailed in the table:

- T_{down} and T_{up} capture the time required to broadcast and upload serialized model parameters over rural WAN links.
- T_{comp} is the time client k spends executing E local epochs of SGD on its local dataset.
- T_{agg} is the coordinator's weighted parameter reduction time.

The Crucial Insight — The Straggler Problem: Notice the mathematical max operator over the set of participating clients $\mathcal{K}_t$. In a synchronous barrier, the entire round cannot advance until the **slowest client** finishes. If Farm A has an NVIDIA GPU and finishes local training in 2 seconds, but Farm C runs on a low-power CPU taking 20 seconds, the coordinator is blocked waiting for Farm C. This demonstrates why client hardware heterogeneity directly impacts distributed throughput and why timeout barriers are essential."

Transition: *"Now, how exactly do we model each farm within distributed systems theory? Let us examine Slide 7."*

---

7 Slide 7: Farm Nodes as Distributed Compute Nodes

Slide Title: *Farm Nodes as Distributed Compute Nodes*
Suggested Timing: 50–60 seconds.

Key Distributed Systems Points to Emphasize

- **Shared-Nothing architecture:** Fully independent memory, storage, and compute.
- **Local state isolation (σ_k):** Strict sovereignty and encapsulation.
- **Independent failure domains (failure isolation):** A crash at Farm A never cascades to Farm B.
- **Horizontal scalability:** Marginal complexity $\mathcal{O}(Kd)$ without dataset re-partitioning.

What to Say in the Presentation (Literal Script)

"In our theoretical framework, each farm is not merely a data source, but an **autonomous distributed compute node** under a strict ***shared-nothing*** architecture:
Fundamental Node Properties:

- **Local State Isolation:** Node k maintains a private state tuple $\sigma_k = (w_k, \mathcal{D}_k, n_k, t_k)$ containing its model weights, dataset, sample count, and local round index. No outside node can read or tamper with its internal memory.
- **Independent Failure Domains:** A power outage or hardware failure at Farm A never compromises the execution of Farm B or C. There is absolute failure isolation across domains.
- **Stateless Interactions:** From the node's vantage point, each RPC iteration is self-contained, enabling clean hot-restarts and node replacements.

Scalability and Relaxed Consensus: Unlike strict consensus protocols such as Raft or Paxos where every write demands synchronized replica locks, federated learning adopts **optimistic state replication**: nodes advance locally in parallel, and consensus is reached post-hoc during weighted aggregation. Furthermore, enrolling a new node $K+1$ scales horizontally in $\mathcal{O}(Kd)$ time without modifying existing datasets."

Transition: *"To govern this autonomous node, we formally define its finite state machine and distributed disconnection scenarios."*

---

8 Slide 8: Node Lifecycle & Distributed Fault Scenarios

Slide Title: *Node Lifecycle & Distributed Fault Scenarios*
Suggested Timing: 55–65 seconds.

Key Distributed Systems Points to Emphasize

- **Finite State Machine (FSM):** IDLE → SYNCHRONIZING → TRAINING → REPORTING → PARTITIONED.
- **Network partitions and stragglers:** Detected via barrier timeout timer τ_{timeout}.
- **Majority quorum aggregation:** $|\mathcal{S}_t| \geq \lceil K/2 \rceil + 1$ to prevent split-brain anomalies.
- **Transparent state catch-up:** Re-synchronization in the subsequent round via direct parameter overwrite.

What to Say in the Presentation (Literal Script)

"This slide details the node lifecycle and distributed fault tolerance mechanisms:
1. Finite State Machine (FSM): Each node transitions deterministically through 5 operational states:

- **IDLE:** Waiting for incoming coordinator round requests.
- **SYNCHRONIZING:** Receives $w^{(t)}$ and overwrites its local replica (state synchronization event).
- **TRAINING:** Computes E local SGD epochs; here local state diverges from the global model (the origin of client drift).
- **REPORTING:** Serializes updated weights $w_k^{(t+1)}$ into a `FitRes` message and transmits them.
- **PARTITIONED:** If network connectivity drops or the host crashes, the node enters a partitioned state.

2. Fault Tolerance Mechanics and Quorum:

- If a node fails to report before the barrier timeout τ_{timeout}, the coordinator marks it as disconnected and **dynamically recalibrates the surviving quorum** $\mathcal{S}_t$.
- We enforce a strict majority quorum $|\mathcal{S}_t| \geq \lceil K/2 \rceil + 1$ (at least 2 out of 3 nodes) to prevent *split-brain* inconsistencies.
- **Transparent Recovery:** When a disconnected node reconnects in a subsequent round, it performs *state catch-up* simply by pulling the latest global model $w^{(t)}$, completely eliminating the need for expensive distributed replay logs."

Transition: *"Let us now examine the end-to-end system architecture and how all these physical components interact across the network topology."*

---

9 Slide 9: Collaborative Farm Network Architecture

Slide Title: *Collaborative Farm Network Architecture*
Suggested Timing: 50–60 seconds.

Key Distributed Systems Points to Emphasize

- **Full topology:** Partitioning engine → Farm Nodes A, B, C → Central Coordinator.
- **Explicit message channels:** `FitIns` (dashed blue broadcast) and `FitRes` (solid black uplink).
- **Distributed decoupling:** Strict boundary between private edge datasets and shared global model parameters.
- **Model consumers:** On-device Edge Inference Engine (Farmer Support Tool) and Distributed Evaluation Suite.

What to Say in the Presentation (Literal Script)

"This architectural diagram summarizes the collaborative distributed network:

- **On the left:** We ingest the benchmark *PlantVillage* dataset, processed by a controlled partitioning engine that generates local data slices for each farm.
- **In the center (The 3 Worker Nodes):**
  - Farm A: Specializing in fungal blights and rusts.
  - Farm B: Dominated by bacterial spots.
  - Farm C: Representing viral mosaic diseases and healthy control foliage.
- **The Federated Coordinator:** Orchestrates barrier synchronization. Dashed blue lines indicate `FitIns` broadcasts containing global model weights; solid black lines show `FitRes` responses transmitting trained tensor updates and sample counts.
- **On the right:** The converged global model serves two endpoints: the centralized evaluation suite and a standalone lightweight edge utility (**Farmer Support Tool**) for offline field diagnosis.

Core takeaway: Privacy is strictly maintained: raw leaf images never touch network links; only numerical parameter updates are exchanged."

Transition: *"To evaluate this architecture with scientific rigor, how do we configure the controlled data partitions, and how does the model calibrate predictions at the edge?"*

---

10 Slide 10: Controlled Non-IID Partitioning & Edge Calibration

Slide Title: *Controlled Non-IID Partitioning & Edge Support Tool*
Suggested Timing: 50–60 seconds.

What to Say in the Presentation (Literal Script)

"This slide highlights two essential methodological components:
1. The Four Controlled Partitioning Regimes: To quantify the precise impact of statistical data skew, we implement 4 reproducible partition setups:

1. **IID Baseline ($\alpha \to \infty$):** Perfectly balanced distributions ($N/3$ samples with identical class proportions across all farms).
2. **Low Non-IID ($\alpha = 1.0$):** Dirichlet distribution introducing mild inter-node class skew.
3. **Moderate Non-IID ($\alpha = 0.5$):** Dirichlet partition where minority classes account for under 10% of local data.
4. **High Non-IID (85/15 Disjoint Rule):** Extreme geographic skew where each farm holds 85% of a specific disease cluster and only 7.5% of the remaining two.

2. Farmer Support Tool and Edge Calibration: At the edge, standard softmax outputs tend to suffer from miscalibration and overconfidence. We implement **Temperature Scaling** and assess the **Expected Calibration Error (ECE)**, ensuring that the farmer receives not merely a raw diagnostic label, but well-calibrated probabilities alongside actionable agronomic advice (such as fungicide treatment or immediate quarantine)."

Transition: *"All of this is systematically evaluated across our 6-track targeted experimental matrix."*

---

11 Slide 11: Targeted Experimental Matrix (6 Key Tracks)

Slide Title: *Targeted Experimental Matrix (6 Systematic Tracks)*
Suggested Timing: 55–65 seconds.

Key Distributed Systems Points to Emphasize

- **Track 4 (Fault tolerance / Node Churn):** Forced crash of Farm C at round $t = 5$ and quorum renegotiation.
- **Track 5 (Communication complexity):** Serialized payload tracking and barrier latency over simulated rural links.
- **Custom stability metric ($S_{\Delta acc}$):** Standard deviation of inter-round accuracy deltas to quantify convergence smoothness.

What to Say in the Presentation (Literal Script)

"Our experimental matrix encompasses six rigorous evaluation tracks:

1. **FedAvg vs. FedProx Optimization:** Comparing convergence rate, global top-1 accuracy, and loss trajectories across rounds.
2. **Impact of Statistical Heterogeneity:** Measuring accuracy degradation as we transition from balanced IID to extreme 85/15 disjoint partitions.
3. **Proximal Parameter Sweep (μ):** Evaluating $\mu \in \{0.001, 0.01, 0.1, 1.0\}$ to pinpoint the sweet spot between drift stability and local learning stagnation.
4. **Fault Tolerance and Node Churn: Our core distributed systems test**: we intentionally disconnect Farm C at round $t = 5$ to assess how the coordinator dynamically renormalizes weights from the surviving quorum without service interruption.
5. **Communication Complexity:** Profiling serialized payload volume in megabytes and round barrier latency.
6. **Fairness Across Nodes and Minority Classes:** Evaluating per-class recall to guarantee that smaller farms do not suffer diagnostic marginalization.

To rigorously quantify optimization stability, we introduce the formal metric $S_{\Delta acc}$ shown in the footer: lower standard deviation in inter-round accuracy increments indicates a smoother, more predictable distributed convergence."

Transition: *"Rather than presenting purely conceptual plans, we have already built and empirically validated our distributed transport prototype. Let us examine the preliminary results on Slide 12."*

---

12 Slide 12: Preliminary Results: RabbitMQ Prototype & Quorum Validation

Slide Title: *Preliminary Results: RabbitMQ Prototype & Quorum Validation*
Suggested Timing: 60–70 seconds.

Key Distributed Systems Points to Emphasize

- **Real distributed implementation:** Developed and benchmarked in Python and RabbitMQ (AMQP 0-9-1) before running heavy CNN training.
- **Complete quorum ($K = 3$, $N = 60$):** Weighted aggregation mathematically verified ($w^{(1)} \approx 2.8333$), with immediate barrier unblocking ($T_{\text{barrier}} \approx 120\,\text{ms}$, $T_{\text{agg}} = 12.5\,\text{ms}$).
- **Fault-tolerant dropout ($K = 2$, Farm C offline):** Coordinator dynamically renormalizes without crashing or blocking ($w^{(1)}_{\text{dropout}} \approx 1.6667$), bounded strictly by the fault timeout ($\tau = 3.00\,\text{s}$, $T_{\text{agg}} = 8.2\,\text{ms}$).
- **Zero message loss in RabbitMQ:** Durable command queues preserve unconsumed instructions for state catch-up upon reconnection.

What to Say in the Presentation (Literal Script)

"On Slide 12, we present the empirical preliminary results obtained from our fully functional distributed prototype:
Prior to training heavy deep learning backbones, we implemented and validated our coordinator-worker harness using **RabbitMQ with AMQP 0-9-1** across three isolated client processes.

1. Nominal Complete Quorum ($K = 3$):

- In the nominal scenario, Farm A ($n = 10, w = 1$), Farm B ($n = 20, w = 2$), and Farm C ($n = 30, w = 4$) report on time across total $N = 60$ samples.
- The coordinator collects all updates, applies sample-weighted FedAvg, and computes the global weights:

$$w^{(1)} = \frac{10(1) + 20(2) + 30(4)}{60} = \frac{170}{60} \approx \mathbf{2.8333}$$

- From a distributed systems perspective, notice the timing: the barrier unblocks immediately in just **120 milliseconds**, and the server executes vector reduction in **12.5 milliseconds** with zero message loss.

2. Node Dropout Resilience ($K = 2$, Farm C down):

- In our second benchmark, we injected a fail-stop network failure on Farm C.
- Instead of deadlocking, the coordinator barrier is strictly bounded by the fault timeout ($\tau = \mathbf{3.00}$ seconds). It detects that the responsive subset meets our minimum quorum threshold ($|\mathcal{S}_t| \geq 2$), and dynamically re-normalizes weights across Farms A and B:

$$w^{(1)}_{\text{dropout}} = \frac{10(1) + 20(2)}{30} = \frac{50}{30} \approx \mathbf{1.6667}$$

- The partial aggregation took only **8.2 milliseconds**. Crucially, RabbitMQ broker inspection confirmed that Farm C's command was not dropped: it remained safely buffered in its durable queue, guaranteeing that when Farm C reconnects, it performs zero-overhead **state catch-up**."

Transition: *"This empirical verification proves our distributed mechanics work in practice. Let us now examine how distributed systems theory directly governs this architecture."*

---

13 Slide 13: Distributed Systems Theory in Federated Agriculture

Slide Title: *Distributed Systems Theory in Federated Agriculture*
Suggested Timing: 60–70 seconds.

Key Distributed Systems Points to Emphasize

- **CAP Theorem (Brewer, 2000):** Architectural design favoring **AP** (Availability + Partition Tolerance) with eventual consistency.
- **RabbitMQ AMQP 0-9-1 Messaging:** Durable exchanges, persistent command queues, manual acks (`FitIns`, `FitRes`).
- **Lamport Causal Ordering ($\prec$):** broadcast $\prec$ train$_k$ $\prec$ upload $\prec$ aggregate.
- **Duplicate Suppression and Idempotent Aggregation:** Unique round identifiers preventing duplicate aggregation bias.

What to Say in the Presentation (Literal Script)

"This slide bridges our empirical design with core theorems of Distributed Systems theory:
1. Brewer's CAP Theorem: Given unavoidable rural network partitions (P), distributed systems must trade off strict consistency (C) against continuous availability (A).

- Our federated network deliberately adopts an **AP (Availability + Partition Tolerance) architecture**.
- Each farm prioritizes local availability to continue diagnostic inference and local model training even in disconnected offline mode.
- Rather than demanding immediate strong consistency (linearizability), the system converges toward **Eventual Consistency** across consecutive federated synchronization rounds.

2. Message Passing, Logical Clocks, and Network Semantics:

- We utilize RabbitMQ (AMQP 0-9-1) with durable exchanges and persistent queues to decouple farm execution from coordinator network sessions.
- Execution follows **Lamport's causal ordering** ($\prec$), strictly observing: broadcast $\prec$ local training $\prec$ upload $\prec$ global aggregation.
- To prevent network retransmissions from duplicating weight updates and biasing aggregation, we enforce **duplicate suppression** with monotonic round IDs, guaranteeing that server aggregation is strictly **idempotent and deterministic**."

Transition: *"Now, let us examine how we translate this architecture into high-performance software and concrete engineering deliverables."*

---

14 Slide 14: Deliverables & Academic Architecture Strategy

Slide Title: *Deliverables & Academic Architecture Strategy*
Suggested Timing: 50–60 seconds.

Key Distributed Systems Points to Emphasize

- **Strict modular separation:** Distributed federated simulation (Python/PyTorch/RabbitMQ) vs. Edge inference engine (C++).
- **C++ Runtime with ONNX Runtime:** Zero Python runtime dependencies on edge hardware; sub-50 ms latency.
- **Open-source and reproducible:** GitHub repository with end-to-end automation scripts.

What to Say in the Presentation (Literal Script)

"Our engineering roadmap cleanly decouples the system lifecycle into two modular components:

1. **Distributed Federated Simulation Pipeline:** Built in **Python, PyTorch, and RabbitMQ (AMQP 0-9-1)**. It manages multi-process worker concurrency, AMQP serialization, barrier timeouts, and the synthetic Non-IID partitioning suite.
2. **High-Performance C++ Edge Diagnostic Engine:**
   - Following federated convergence, the global model is exported and compiled into optimized formats: **ONNX and TorchScript**.
   - The model executes inside a native binary utilizing the **ONNX Runtime C++ API**.
   - This completely strips away the Python runtime from field devices, allowing deployment on resource-constrained hardware such as Raspberry Pi boards or IoT agricultural gateways with deterministic latencies under **50 milliseconds**.

All codebase components, experiment configurations, and reproduction pipelines are versioned and openly accessible on GitHub."

Transition: *"To ensure complete project feasibility, let us look at our 14-week execution schedule and open reproducibility protocol on Slide 15."*

---

15 Slide 15: Project Feasibility, Work Schedule & Reproducibility

Slide Title: *Project Feasibility, Work Schedule & Reproducibility*
Suggested Timing: 50–60 seconds.

Key Points to Emphasize

- **14-week timeline structured in 6 phases:** Phases 1 and 2 completed; Phase 3 currently in progress.
- **Open-science reproducibility:** Public GitHub repository with deterministic seeds (`seed=42`) and automated unit test suite.

What to Say in the Presentation (Literal Script)

"On Slide 15, we demonstrate the concrete feasibility and reproducibility of our research:

1. **14-Week Milestone Timeline:** The project is organized into 6 clearly bounded phases:
   - **Phase 1 (AMQP Prototype) and Phase 2 (Dirichlet Sharding)** are already completed and verified.
   - We are currently in **Phase 3**, integrating the PyTorch FedAvg and FedProx training loops.
   - The upcoming weeks cover the systematic execution of our 6 experimental tracks, edge ONNX compilation, and final camera-ready report authoring.
2. **Feasibility and Open-Science Reproducibility:**
   - **Code Availability:** All code, configs, and unit tests are publicly hosted at `https://github.com/darioXxx32/IA`.
   - **Deterministic Execution:** All pseudo-random generators use fixed seeds (`seed=42`), enabling complete one-command verification via `test_protocol.py` and `run_demo.py`."

Transition: *"What are the immediate milestones to complete the final scientific report? Let us review the roadmap on Slide 16."*

---

16 Slide 16: Expected Contribution & Roadmap for Final Report

Slide Title: *Roadmap for Final Report & Expected Contributions*
Suggested Timing: 45–55 seconds.

What to Say in the Presentation (Literal Script)

"Our roadmap toward the final scientific report spans five concrete milestones:

1. Execute the complete 6-track experimental matrix across the *PlantVillage* partitions across all Non-IID regimes.
2. Finalize the systematic μ parameter sweep over $\{0.001, 0.01, 0.1, 1.0\}$, compiling Macro-F1 scores, per-node confusion matrices, and the $S_{\Delta acc}$ stability metric.
3. Conduct the node churn fault injection experiment at round $t = 5$ to validate quorum rebalancing and consensus resilience.
4. Compile the standalone C++ ONNX Runtime binary and polish the interactive command-line interface (CLI) for field agronomists.
5. Author the final 8-to-15-page scientific paper adhering to required MDPI journal formatting standards (*Agronomy / Applied Sciences*).

Our Primary Contribution: To formally demonstrate how distributed systems principles combined with FedProx's proximal regularization simultaneously solve data sovereignty, bandwidth bottlenecks, and diagnostic fairness in highly fragmented agricultural settings."

Transition: *"With this, I conclude my presentation and open the floor for discussion."*

---

17 Slide 17: Conclusion & Q&A

Slide Title: *Conclusion*
Suggested Timing: 35–45 seconds.

Strong Closing and Opening for Questions (Literal Script)

"In conclusion:
This project demonstrates that Federated Learning not only protects commercial data privacy and saves over 98% of rural network bandwidth, but—by incorporating FedProx's elastic consensus and the partition tolerance of the CAP theorem—enables agricultural cooperatives across distinct geographic realities to collaborate as a truly resilient, equitable distributed system.
Thank you very much for your time and attention. I am now open to your questions and feedback to enrich this project."

Tips for the Q&A Session

- **If they ask about RabbitMQ:** Highlight that RabbitMQ with AMQP 0-9-1 provides durable message queues and manual acknowledgments, buffering round instructions for offline farms and decoupling local training time from coordinator network sessions.
- **If they ask why 3 nodes:** Explain that 3 nodes represents the canonical minimal setup to model geographic diversity (Valley, Highland, Lowland), investigate majority quorum consensus (2/3), and withstand a single node crash without triggering symmetric network partitioning.
- **If they ask why C++ for the edge:** Emphasize energy efficiency, minimal RAM footprint, and deterministic sub-50ms latency on low-power IoT gateways where a Python runtime is too resource-heavy.
- **If they ask about Byzantine Fault Tolerance:** Clarify that this proposal assumes an honest crash-stop failure model, and that Byzantine-robust aggregators (such as Krum or Trimmed-Mean) constitute promising future research extensions.